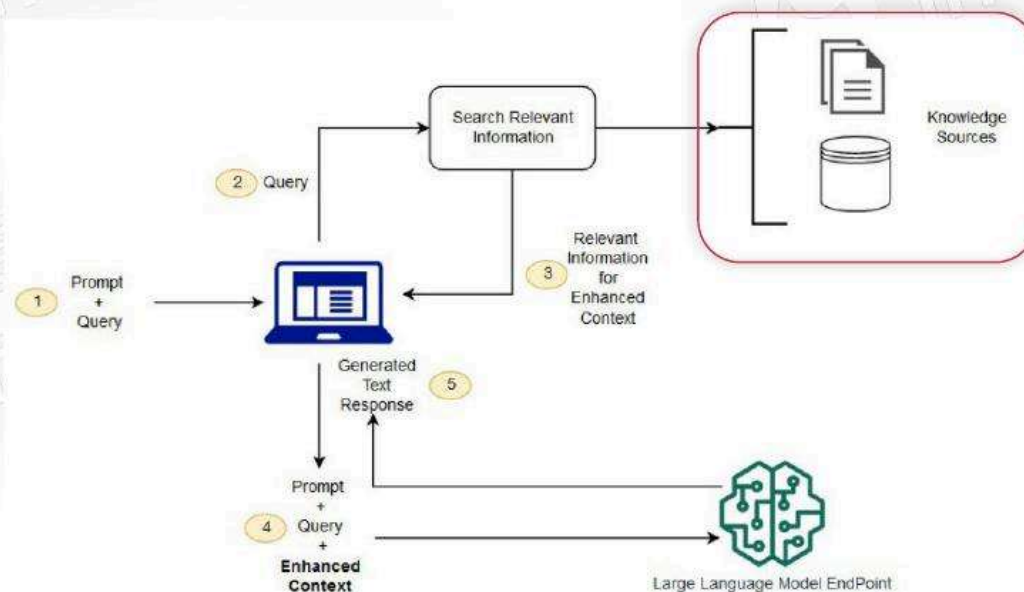


garbage in garbage out
企业复杂业务数据智能处理

本章简介

- ◆ 知识库为RAG流程提供最重要的上下文信息，而如何高效准确的处理企业数据是构建知识库的关键



本章简介

◆ 企业数据复杂多样

◆ 文档分块

◆ 垃圾进垃圾出原则

◆ 实战



◆ 读取多样性文档

企业数据复杂多样

◆ 企业数据复杂多样：格式复杂



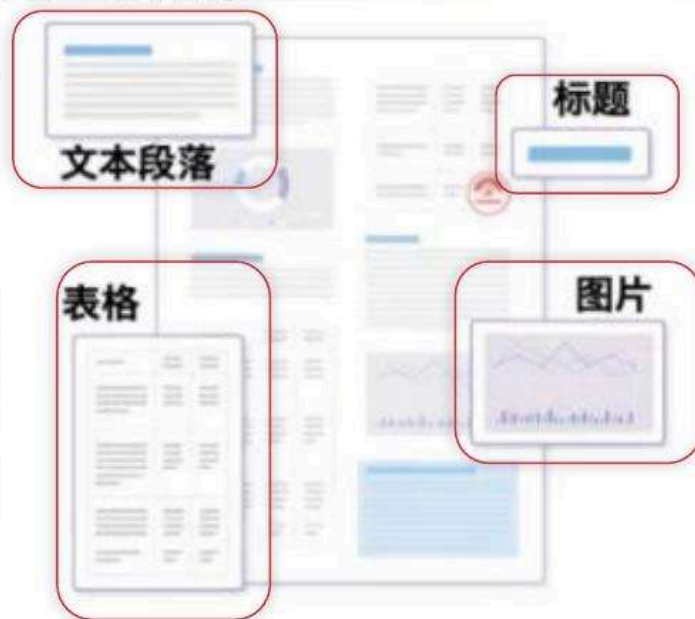
非结构或半结构化数据



结构化数据

企业数据复杂多样

◆ 企业数据复杂多样：内容复杂



$$e^{i\pi} + 1 = 0$$

$e^{ix} = \cos(x) + i\sin(x)$

企业数据复杂多样

◆ 企业数据复杂多样：内容复杂

**Ancient Document Layout Analysis:
Autoencoders meet Sparse Coding**

Piero Davoli, Marco Fornaci, Arianna Trivella
Center for Cultural Heritage Technology (CCHT), Institute of Technology (IT)
Email: piero.davoli, marco.fornaci, arianna.trivella@uniroma1.it

Historical Document Layout Analysis

- Segmenting image into homogeneous regions
 - Block of text, side notes, drawings, tables, etc.
- Key preprocessing step in various applications
- Still an open problem for historical documents
 - Usually lacking a structured text management
 - High degradation
- Deep Neural Networks for Document Analysis
 - specific classification methods
 - feature learning based methods
- A novel unsupervised representation learning method for DLA
 - Based on the sparse representation of image patches

DLA pipeline

- Four key patches extracted
- Sparse representation vector computed for each extracted patch
- A text forward network trained to classify each patch

Neural Sparse Coding

- Classical sparse coding
 - restricted to the linear combination of sparse feature vector and dictionary terms
- Recent DNN based sparse coding
 - Train the encoder in supervised way
 - modeling the hierarchical mixture steps via unfolding a neural network
- Our method
 - Encoder and sparse representations are trained simultaneously in an end-to-end fashion

Sparse Representation Learning

- Architecture
 - Encoder: Encoder with sparse latent variables

Classical Cost: $\|x - \sum_{i=1}^K w_i \phi_i\|_2^2$

Regularized Cost: $\|x - \sum_{i=1}^K w_i \phi_i\|_2^2 + \lambda \sum_{i=1}^K \|w_i\|_1$

Training

- Supervised learning
 - Neural network steps: no coding, but encoding step (from the learned dictionary Φ)
 - We skip the dictionary learned by VQ-RLS
- Unsupervised learning
 - Inspired by the VQ-RLS algorithm, sparse encoder network is trained in an iterative way

Experiments and Results

- Experiments on DNA-HuB dataset [6]

Reference

[1] P. Davoli, M. Fornaci, A. Trivella, "Ancient Document Layout Analysis: Autoencoders meet Sparse Coding", in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2019, pp. 1000-1009.

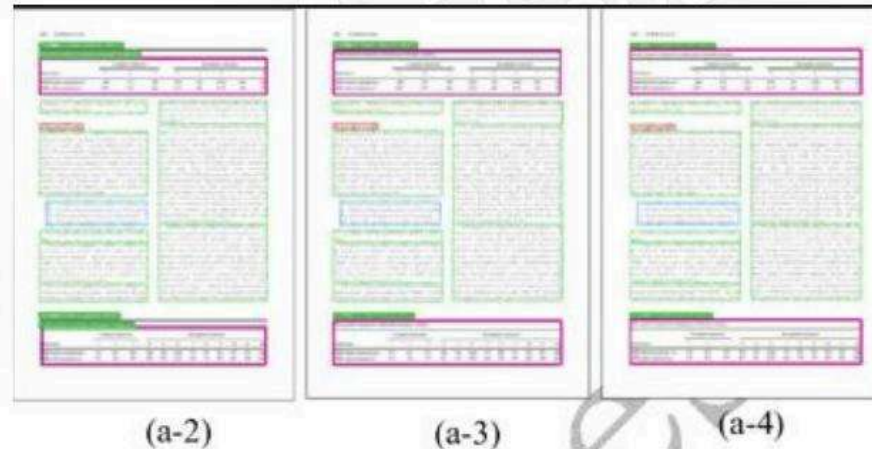
[2] P. Davoli, M. Fornaci, A. Trivella, "Ancient Document Layout Analysis: Autoencoders meet Sparse Coding", in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2019, pp. 1000-1009.

[3] P. Davoli, M. Fornaci, A. Trivella, "Ancient Document Layout Analysis: Autoencoders meet Sparse Coding", in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2019, pp. 1000-1009.

[4] P. Davoli, M. Fornaci, A. Trivella, "Ancient Document Layout Analysis: Autoencoders meet Sparse Coding", in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2019, pp. 1000-1009.

[5] P. Davoli, M. Fornaci, A. Trivella, "Ancient Document Layout Analysis: Autoencoders meet Sparse Coding", in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2019, pp. 1000-1009.

[6] P. Davoli, M. Fornaci, A. Trivella, "Ancient Document Layout Analysis: Autoencoders meet Sparse Coding", in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2019, pp. 1000-1009.



(a)

Text Title List Table Figure

垃圾进垃圾出-注重文档质量

◆ 原则：垃圾进垃圾出GIGO



如果输入数据质量很差，例如包含相互冲突的信息，数据解析数据分割存在错误，那么无论您将 RAG 系统构建得多么好，都无法产生应有的价值

垃圾进垃圾出-注重文档质量

◆ 原则：垃圾进垃圾出GIGO

垃圾数据可能得原因

源数据：针对RAG应用选择的数据问题
(不相关 / 存在矛盾 / 不全面)

数据解析：内容识别错误，关键词、句子或段落被误读或遗漏；表格的行列关系混乱等

数据分块：同一个语义的分割成不同块或者不同语义分在同一个块，造成干扰

RAG如何读取多样性文档

◆ 文档解析工具



LangChain

Document loaders

Beautiful Soup

python-docx

python—pptx



LlamaIndex

SimpleDirectoryReader

PyMuPDF

openpyxl



RAGFlow

RAG如何读取多样性文档

◆ 文档解析工具-HTML



```
<!doctype html>
<html lang="en">
  <head>
    <meta charset="utf-8">
    <meta name="viewport" content="width=device-width, initial-scale=1">
    <title>Bootstrap demo</title>
  </head>
  <body>
    <h1>Hello, world!</h1>
  </body>
</html>
```


RAG如何读取多样性文档

◆ 文档解析工具-HTML

处理 URL
和 HTTP 请
求

解析html内容



Document Loader	Description	Package/API
Web	Uses <code>urllib</code> and <code>BeautifulSoup</code> to load and parse HTML web pages	Package
RecursiveURL	Recursively scrapes all child links from a root URL	Package
Sitemap	Scrapes all pages on a given sitemap	Package
Firecrawl	API service that can be deployed locally, hosted version has free credits.	API

Langchain Document loaders

https://python.langchain.com/v0.2/docs/integrations/document_loaders/

RAG如何读取多样性文档

◆ 文档解析工具-HTML

```
from langchain_community.document_loaders import WebBaseLoader

loader = WebBaseLoader("https://www.espn.com/")
```

```
from langchain_community.document_loaders import BSHTMLLoader

loader = BSHTMLLoader(
    file_path="./example_data/fake-content.html",
)
```

Beautiful Soup
用于从 HTML 和 XML 文件中
提取数据

只提取文本信息，没有特殊标记内容如代码模块

RAG如何读取多样性文档

◆ 文档解析工具-HTML

Create a project directory and enter it:

```
$ mkdir flask-tutorial  
$ cd flask-tutorial
```

Then follow the [installation instructions](#) to set up a Python virtual environment and install Flask for your project.

The tutorial will assume you're working from the `flask-tutorial` directory from now on. The file names at the top of each code block are relative to this directory.

A Flask application can be as simple as a single file.

```
hello.py  
  
from flask import Flask  
  
app = Flask(__name__)  
  
@app.route('/')  
def hello():  
    return 'Hello, World!'
```

```
> <div class="related" role="navigation" aria-label="Related"></div>  
▼ <div class="document">  
  > <div class="documentwrapper">  
    > <div class="bodywrapper">  
      > <div class="body" role="main">  
        > <section id="project-layout">  
          > <h1></h1>  
          > <p>Create a project directory and enter it:</p>  
          ▼ <div class="highlight-none notranslate">  
            > <div class="highlight">  
              > <pre> == $  
                <span></span>  
                > $ mkdir flask-tutorial $ cd flask-tutorial *  
              </pre>  
            </div>  
          </div>  
          > <p></p>  
          > <p></p>  
          > <hr class="docutils">  
          > <p>A Flask application can be as simple as a single file.</p>  
          > <div class="literal-block-wrapper docutils container" id="id1"></div>
```

Beautiful Soup

RAG如何读取多样性文档

◆ 文档解析工具-HTML

```
from bs4 import BeautifulSoup
soup = BeautifulSoup(response.text, 'lxml')

description_elements = soup.find_all('div', class_='highlight')

for element in description_elements:
    print(element.text)
    print("="*100)
```

```
$ mkdir flask-tutorial
$ cd flask-tutorial

=====

from flask import Flask

app = Flask(__name__)

@app.route('/')
def hello():
    return 'Hello, World!'
```

Beautiful Soup

RAG如何读取多样性文档

◆ 文档解析工具-PDF



Adobe公司开发的文档格式



Semi-structured document: html

```
</DOCTYPE html>
<html>
<body>
<p> paragraph </p>
<table border="1">
  <tr>
    <td>Row 1, cell 1</td>
    <td>Row 1, cell 2</td>
  </tr>
</table>
</body>
</html>
```

Unstructured document: PDF

```
4 0 obj
<< >>
stream
1. 0. 0. 1. 50. 300. cm
BT
/F0 26. Tf
(Hello, World!) Tj
ET
endstream
endobj
```

打印指令的集合

RAG如何读取多样性文档

◆ 文档解析工具-PDF



Document Loader	Description	Package/API
PyPDF	Uses 'pypdf' to load and parse PDFs	Package
Unstructured	Uses Unstructured's open source library to load PDFs	Package
Amazon Textract	Uses AWS API to load PDFs	API
MathPix	Uses MathPix to load PDFs	Package
PDFPlumber	Load PDF files using PDFPlumber	Package
PyPDFDirectory	Load a directory with PDF files	Package
PyPDFium2	Load PDF files using PyPDFium2	Package
UnstructuredPDFLoader	Load PDF files using Unstructured	Package
PyMuPDF	Load PDF files using PyMuPDF	Package
PDFMiner	Load PDF files using PDFMiner	Package

正确保留双列布局
的文本顺序，同时
能提取表格和图片

Langchain Document loaders

https://python.langchain.com/v0.2/docs/integrations/document_loaders/

RAG如何读取多样性文档

◆ 文档解析工具-PDF

```
from langchain_community.document_loaders import PyMuPDFLoader

loader = PyMuPDFLoader("./example_data/layout-parser-paper.pdf")
```

RAG如何读取多样性文档

◆ 文档解析工具-PDF

```
import fitz
doc = fitz.open("../data/zhidu_travel.pdf")

table_data = []
text_data = []

doc_tables = {}
for idx, page in enumerate(doc):
    text = page.get_text()
    text_data.append(text)
    tabs = page.find_tables()
    for i, tab in enumerate(tabs):
        ds = tab.to_pandas()
        table_data.append(ds.to_markdown())
```

直接使用pymuPDF

提取表格

RAG如何读取多样性文档

◆ 文档解析工具-PDF

PP-Structure

飞桨



PDF 数据提取神器

PDFlux



Miner U



RAGFlow

LlamaParse

RAG如何读取多样性文档

◆ 文档解析工具



Document Loader	Data Type
CSVLoader	CSV files
DirectoryLoader	All files in a given directory
Unstructured	All file types
JSONLoader	JSON files
UnstructuredMarkdownLoader	Markdown files
BSHTMLLoader	HTML files
UnstrucutredXMLLoader	XML files

支持所有文件类型



Langchain Document loaders

https://python.langchain.com/v0.2/docs/integrations/document_loaders/

RAG如何读取多样性文档

◆ 文档解析工具-office



```
from langchain_community.document_loaders import UnstructuredExcelLoader
```

```
loader = UnstructuredExcelLoader("./example_data/stanley-cups.xlsx", mode="elements")
```



```
from langchain_community.document_loaders import UnstructuredPowerPointLoader
```

```
loader = UnstructuredPowerPointLoader("./example_data/fake-power-point.pptx")
```



```
from langchain_community.document_loaders import Docx2txtLoader
```

```
loader = Docx2txtLoader("./example_data/fake.docx")
```

Langchain Document loaders

https://python.langchain.com/v0.2/docs/integrations/document_loaders/

RAG如何读取多样性文档

更强的定制化能力

◆ 文档解析工具-office



openpyxl
python-pptx
python-docx

```
from openpyxl import load_workbook
```

```
# 打开现有的工作簿
```

```
wb = load_workbook('example.xlsx')
```

```
from pptx import Presentation
```

```
prs = Presentation(path_to_presentation)
```

```
from docx import Document
```

```
def read_docx(file_path):
```

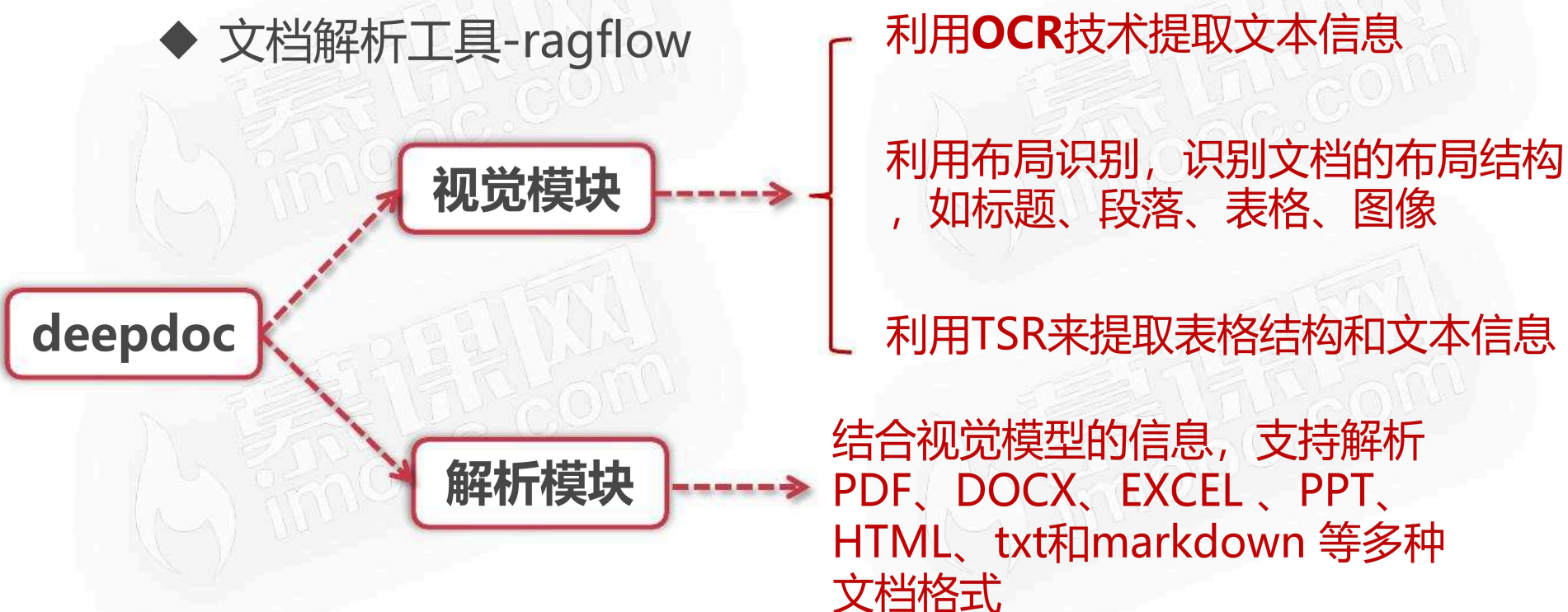
```
    doc = Document(file_path)
```

```
    for para in doc.paragraphs:
```

```
        print(para.text)
```

RAG如何读取多样性文档

◆ 文档解析工具-ragflow



RAGFlow: deepdoc
<https://github.com/infiniflow/ragflow>

RAG如何读取多样性文档

◆ 文档解析工具-ragflow

ocr检测和识别文本

```
python deepdoc/vision/t_ocr.py -h
usage: t_ocr.py [-h] --inputs INPUTS [--output_dir OUTPUT_DIR]

options:
  -h, --help            show this help message and exit
  --inputs INPUTS       Directory where to store images or PDFs, or a file path to a single image or PDF
  --output_dir OUTPUT_DIR
                        Directory where to store the output images. Default: './ocr_outputs'
```

Abstract

In this note, we present new zero-knowledge interactive proofs and arguments for languages in NP . To show that $x \in L$, with an error probability of at most 2^{-k} , our zero-knowledge proof system requires $O(|x|^{c_1}) + O(\lg^{c_2} |x|)k$ ideal bit commitments, where c_1 and c_2 depend only on L . This construction is the first

Introduction.

The problem of efficient security amplification.

The standard definition of interactive proofs [7] requires that the verifier accept a correct proof and reject an incorrect assertion with probability at least $\frac{2}{3}$. As there are few applications where a $1/3$ error probability is acceptable, one usually tries to obtain an error probability

RAG如何读取多样性文档

◆ 文档解析工具-ragflow

布局识别

```
python deepdoc/vision/t_recognizer.py -h
```

```
usage: t_recognizer.py [-h] --inputs INPUTS [--output_dir OUTPUT_DIR] [--threshold THRESHOLD] [--mode {layout,tsr}]
```

options:

```
-h, --help            show this help message and exit
--inputs INPUTS        Directory where to store images or PDFs, or a file path to a single image or PDF
--output_dir OUTPUT_DIR
                        Directory where to store the output images. Default: './layouts_outputs'
--threshold THRESHOLD
                        A threshold to filter out detections. Default: 0.5
--mode {layout,tsr}    Task mode: layout recognition or table structure recognition
```

• Q : the (normalized) eigenvectors of the matrix $A^T A$ (i.e., $Q^T Q = I$). The columns of Q are called the *right singular vectors* of A .

• Δ : the diagonal matrix of the *singular values*, $\Delta = A^T A$ with A being the diagonal matrix of the eigenvalues of matrix AA^T and of the matrix $A^T A$ (they are the same).

The SVD is a consequence of the eigen-decomposition of a positive semi-definite matrix. This can be shown by considering the eigen-decomposition of the two positive semi-definite matrices that can be obtained from A : namely AA^T and $A^T A$. If we express these matrices in terms of the SVD of A , we obtain the following equations:

$$AA^T = PAQ^T Q \Delta P^T = P \Delta^2 P^T = PAP^T, \quad (3)$$

and

$$A^T A = Q \Delta P^T P \Delta Q^T = Q \Delta^2 Q^T = Q \Lambda Q^T. \quad (4)$$

Table 1. Diagnostic Performance Characteristics of PSA Testing, According to Cutoff Level.

Characteristic	PSA Cutoff Level	
	≥ 4.0 ng/ml	< 4.0 ng/ml
Test positivity	32	18
Cancer-detection rate	3	4
Sensitivity	21	32
Specificity	91	85
Positive predictive value	30	25

* Data are from Wolf et al.¹⁰ PSA denotes prostate-specific antigen.

patterns or grades in a prostate tumor, each of which is graded on a scale of 1 to 5, with 5 indicating the most aggressive pattern. Nonetheless, prostate-cancer mortality was not reduced in the screening group as compared with the control group (rate ratio, 1.1; 95% CI, 0.75 to 1.70).

However, several factors could have biased the results of the PLCO trial toward the null hypothesis. More than 40% of enrolled subjects had undergone at least one PSA test in the 3 years before study enrollment. Serial PSA testing is associated with reduced rates of prostate-cancer detection as well as an earlier stage and less aggressive tumor characteristics at the time of diagnosis.¹⁰ Given the

aggressive treatment of these cancers is associated with unnecessary risks of urinary, sexual, and bowel dysfunction, which can adversely affect the quality of life.²⁴

RELEVANCE TRIALS

Paradoxically, PSA testing became widespread before any data supported the benefit of aggressively treating early-stage cancer. In 2002, the Scandinavian Prostate Cancer Group Study Number 4, which randomly assigned 695 men younger than 75 years of age who had early-stage prostate cancer to radical prostatectomy or watchful waiting, showed a relative hazard reduction for deaths from prostate cancer of 50% among those assigned to prostatectomy (4.6% vs. 8.9%), during a median follow-up of 6.2 years.²⁵ The mortality benefit persisted through 15 years of follow-up.²⁶ However, no survival benefit was seen for men who were older than 65 years of age at the time of diagnosis and treatment. Since only about 5% of the tumors were detected by screening, and more than 75% were palpable, it is questionable whether these results are applicable to patients in the United States.

The Prostate Cancer Intervention versus Observation Trial (NCT00007644) randomly assigned 774 men with early-stage prostate cancer to either radical prostatectomy or watchful waiting.²⁶ Three months of cancers were diagnosed primarily on the basis of abnormal PSA values, and about half were

RAG如何读取多样性文档

◆ 文档解析工具-ragflow

表格解析识别

```
python deepdoc/vision/t_recognizer.py -h
```

```
usage: t_recognizer.py [-h] --inputs INPUTS [--output_dir OUTPUT_DIR] [--threshold THRESHOLD] [--mode {layout,tsr}]
```

options:

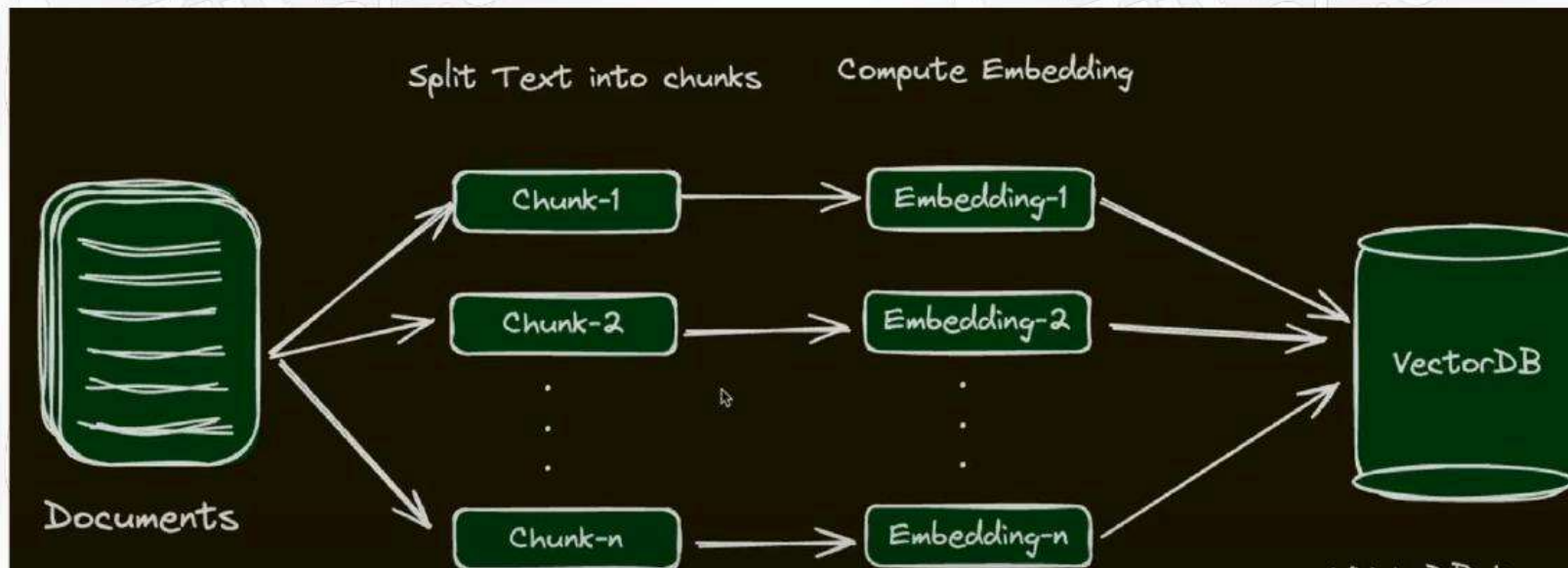
```
-h, --help            show this help message and exit
--inputs INPUTS        Directory where to store images or PDFs, or a file path to a single image or PDF
--output_dir OUTPUT_DIR
                        Directory where to store the output images. Default: './layouts_outputs'
--threshold THRESHOLD
                        A threshold to filter out detections. Default: 0.5
--mode {layout,tsr}    Task mode: layout recognition or table structure recognition
```

Code	Head of Expenditure	Operating Revenue			Investment and Interest Income	Capital Receipts	Total Receipts	Main Estimates				Development Estimates	Total
		Tax Revenue	Fees and Charges	Others				Running Costs	Transfers	Other Outlays	Total		
		\$	\$	\$	\$	\$	\$	\$	\$	\$	\$	\$	\$
B	Attorney-General's Chambers	0	3,583,000	0	3,583,000	0	0	3,583,000					
C	Auditor-General's Office	0	3,578,000	0	3,578,000	0	0	3,578,000					
E	Judicature	0	130,810,000	0	130,810,000	0	0	130,810,000					
F	Parliament	0	689,000	0	689,000	0	0	689,000					
I	Ministry of Social and Family Development	0	9,486,000	0	9,486,000	0	0	9,486,000					
J	Ministry of Defence	0	46,160,000	0	46,160,000	430,000	1,600,000	48,190,000					
K	Ministry of Education	0	350,921,000	0	350,921,000	0	0	350,921,000					
L	Ministry of Sustainability and the Environment	564,775,000	776,784,000	0	1,341,559,000	0	5,000,000	1,346,559,000					

Code	Head of Expenditure	Main Estimates				Development Estimates	Total
		Running Costs	Transfers	Other Outlays	Total		
A	Civil List for the President of the Republic of Singapore	12,235,300	0	0	12,235,300	0	12,235,300
B	Attorney-General's Chambers	262,780,300	19,700	100,000	262,900,000	6,200,000	269,100,000
C	Auditor-General's Office	44,537,900	10,000	0	44,547,900	864,000	45,411,900
D	Cabinet Office	1,100,000	0	0	1,100,000	2,000,000	3,100,000
E	Judicature	376,732,500	0	0	376,732,500	49,710,500	426,443,000
F	Parliament	49,121,900	345,800	0	49,467,700	6,676,000	56,143,700
G	Presidential Councils	1,289,800	0	0	1,289,800	0	1,289,800
H	Public Service Commission	1,876,800	0	0	1,876,800	0	1,876,800
I	Ministry of Social and Family Development	773,158,700	3,787,715,400	5,022,800	4,565,896,900	124,079,000	4,689,975,900
J	Ministry of Defence	19,332,939,900	11,241,700	20,737,200	19,364,918,800	1,440,390,200	20,805,309,000
K	Ministry of Education	9,919,855,900	4,212,144,100	195,475,900	14,327,475,900	620,000,000	14,947,475,900
L	Ministry of Sustainability and the Environment	2,822,305,400	11,939,200	34,000	2,834,278,600	562,239,100	3,426,517,700

文档分块chunk

- ◆ 文档分块：将长文本拆分成更小块的一个过程



文档分块chunk

◆ 文档为什么要分块

原因1



长文档内容包含多个不同语义的信息，对构建embedding语义向量造成干扰，检索匹配精度下降

原因2



检索的长文本信息包含与检索问题不相关的信息，影响LLM生成答案准确性

原因3



向量embedding模型的输入长度有限，需要进行分割

文档分块chunk

◆ 理想的分块chunk

标准



一个chunk表示一个完整且语义相关的上下文信息

确定语料库中文档的最佳块大小对提高搜索结果准确性和相关性是非常重要的；

重要性

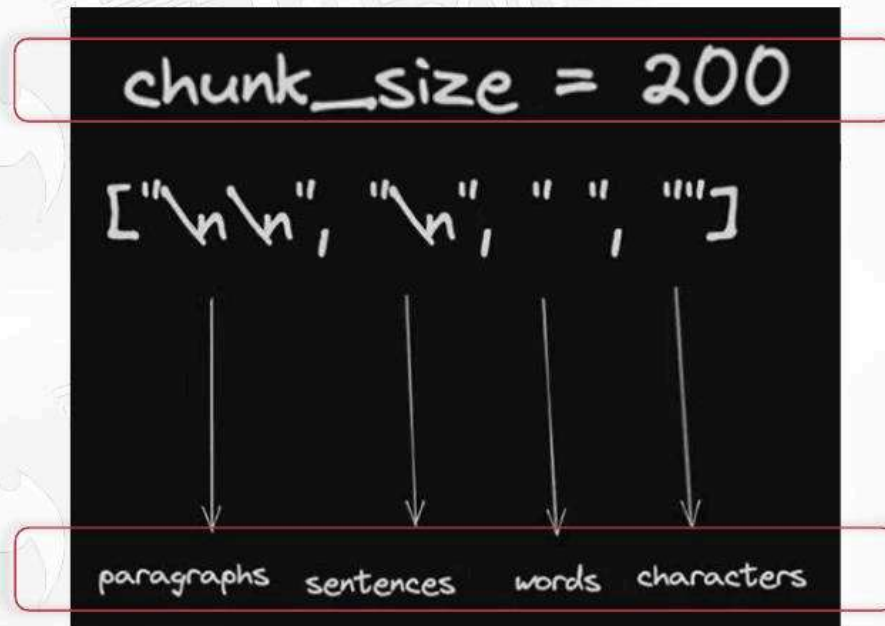


块太小会将原本完整的语义信息分割成多块，导致上下文信息不完整；

块太大就引入不相关的上下文信息，可能导致搜索结果不准确

文档分块chunk

◆ 常用分块chunk方法-RecursiveCharacterTextSplitter



1. 指定块大小chunk_size

2. 指定粗到细的分隔符，从段落/句子/单词/字母

3. 先用粗的分割符拆分文本，如果小于chunk_size则结束；如果大于chunk_size，则依次细的分割符继续拆分，直到满足chunk_size

RecursiveCharacterTextSplitter

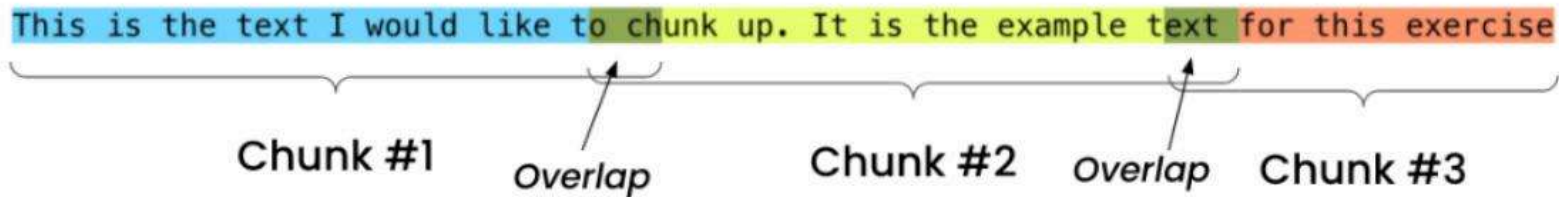
https://python.langchain.com/v0.2/docs/how_to/recursive_text_splitter/

文档分块chunk

◆ 常用分块chunk方法-RecursiveCharacterTextSplitter

```
from langchain_text_splitters import RecursiveCharacterTextSplitter
```

```
# Load example document  
with open("state_of_the_union.txt") as f:
```



```
)  
texts = text_splitter.create_documents([state_of_the_union])  
print(texts[0])  
print(texts[1])
```

RecursiveCharacterTextSplitter

https://python.langchain.com/v0.2/docs/how_to/recursive_text_splitter/

文档分块chunk

◆ 常用分块chunk方法- RecursiveCharacterTextSplitter

```
text_splitter = RecursiveCharacterTextSplitter(  
    separators=[  
        "\n\n",  
        "\n",  
        ",",  
        ".",  
        "!",  
        "?",  
        "\u200b", # Zero-width space  
        "\uff0c", # Fullwidth comma  
        "\u3001", # Ideographic comma  
        "\uff0e", # Fullwidth full stop  
        "\u3002", # Ideographic full stop  
    ],  
    # Existing args  
)
```

中文增加句号和.来表示句子

RecursiveCharacterTextSplitter

https://python.langchain.com/v0.2/docs/how_to/recursive_text_splitter/

文档分块chunk

◆ 特定格式chunk方法-MarkdownTextSplitter

Mayer

For the long-form writer, established blogger, and inspired author.

Features

- * Responsive / Mobile Ready
- * Internationalized
- * Social Networks (Facebook, Pinterest, Email, Twitter, Google+)
- * Readability
- * Editor Styles
- * Personalized For The Author
- * Theme Customizer / No Options
- * Navigation / Simplicity / Less for the user to consider
- * Write. Install / Write

Testimonials

- * TODO

Pricing

- * TODO

Pages

- * The Blog
- * About (Pressware / Tom McFarlin)
- * Documentation / Manual

Mayer

For the long-form writer, established blogger, and inspired author.

Features

- Responsive / Mobile Ready
- Internationalized
- Social Networks (Facebook, Pinterest, Email, Twitter, Google+)
- Readability
- Editor Styles
- Personalized For The Author
- Theme Customizer / No Options
- Navigation / Simplicity / Less for the user to consider
- Write. Install / Write

Testimonials

- TODO

Pricing

- TODO

Pages

文档分块chunk

◆ 特定格式chunk方法-MarkdownTextSplitter

继承RecursiveCharacterTextSplitter

```
from langchain.text_splitter import MarkdownTextSplitter
```

```
splitter = MarkdownTextSplitter(chunk_size = 40, chunk_overlap=0)
```

Separators:

- `\n#{1,6}` - Split by new lines followed by a header (H1 through H6)
- ````\n` - Code blocks
- `\n\\*\\*\\*\\*+\\n` - Horizontal Lines
- `\n---+\\n` - Horizontal Lines
- `\n__+\\n` - Horizontal Lines
- `\n\\n` - Double new lines
- `\n` - New line
- `" "` - Spaces
- `"` - Character

文档分块chunk

◆ 特定格式chunk方法-PythonCodeTextSplitter

继承RecursiveCharacterTextSplitter

```
from langchain.text_splitter import PythonCodeTextSplitter
```

```
python_text = """
class Person:
    def __init__(self, name, age):
        self.name = name
        self.age = age

p1 = Person("John", 36)

for i in range(10):
    print (i)
"""
```

```
python_splitter = PythonCodeTextSplitter(chunk_size=100, chunk_overlap=0)
```

- `\nclass` - Classes first
- `\ndef` - Functions next
- `\n\tdef` - Indented functions
- `\n\n` - Double New lines
- `\n` - New Lines
- `" "` - Spaces
- `"""` - Characters

PythonCodeTextSplitter

文档分块chunk

◆ 智能语义分块

基于规则的分块 ----->

RecursiveCharacterTextSplitter / 特定格式chunk方法

基于语义的分块 ----->

根据文本中的语义信息将文本分成有意义的片段或块的过程，希望每个数据块所包含的信息在语义上相对独立，以便更好地进行检索匹配

文档分块chunk

◆ 智能语义分块-基于embedding语义分块

- 1.将文本按照句子粒度进行分割,
- 2.设置滑动窗口长度
- 3.分别计算前后两个窗口文本的embedding相似度
- 4.相邻且符合阈值的句子会被归入一个语义块

基于embedding相似性的语义分块

https://python.langchain.com/v0.2/docs/how_to/semantic-chunker/

文档分块chunk

◆ 智能语义分块-基于embedding语义分块

embedding模型

分割阈值策略

percentile: 百分位

standard_deviation: 标准差

Interquartile: 四分位

```
text_splitter = SemanticChunker(  
    OpenAIEmbeddings(), breakpoint_threshold_type="percentile"  
)
```

```
docs = text_splitter.create_documents([state_of_the_union])  
print(docs[0].page_content)
```

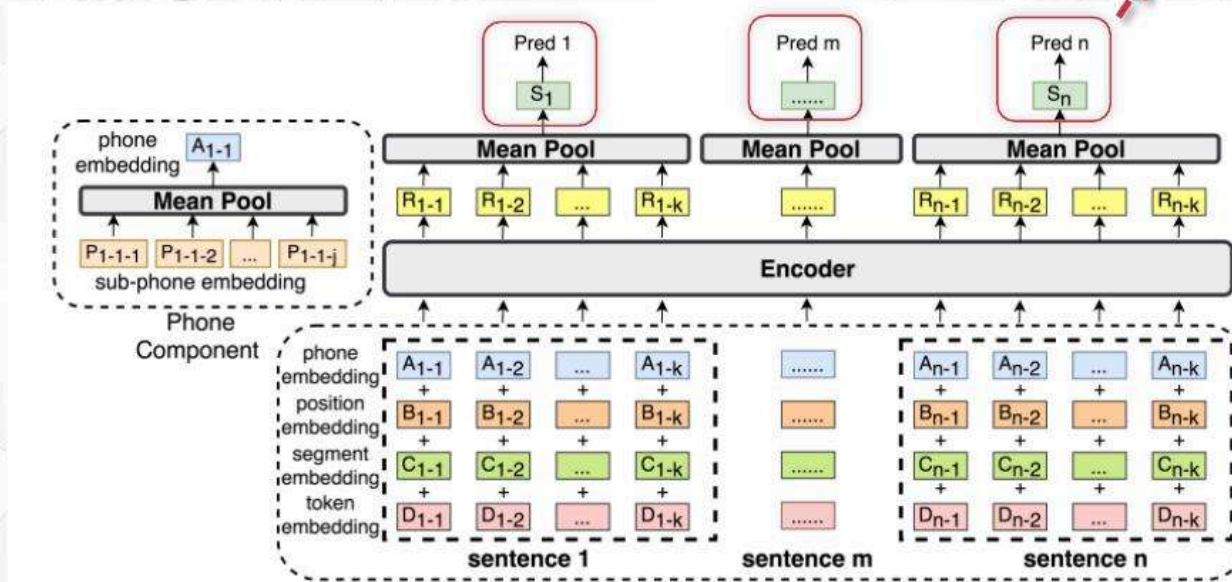
基于embedding相似性的语义分块

https://python.langchain.com/v0.2/docs/how_to/semantic-chunker/

文档分块chunk

◆ 智能语义分块-基于模型的语义分块

预测是否是分割点



基于模型的语义分块

Sequence Model with Self-Adaptive Sliding Window for Efficient Spoken Document Segmentation

文档分块chunk

◆ 智能语义分块-基于模型的语义分块

```
from modelscope.outputs import OutputKeys
from modelscope.pipelines import pipeline
from modelscope.utils.constant import Tasks
```

```
p = pipeline(
    task=Tasks.document_segmentation,
    model='damo/nlp_bert_document-segmentation_chinese-base')
```

result = p(documents='移动端语音唤醒模型，检测关键词为“小云小云”。模型主体为4层FSMN结构，使用CTC训练准则，参数量750K，适用于移动端设备运行。模型输入为Fbank特征，输出为基于char建模的中文全集token预测，测试工具根据每一帧的预测数据进行后处理得到输入音频的实时检测结果。模型训练采用“basetrain + finetune”的模式，basetrain过程使用大量内部移动端数据，在此基础上，使用1万条设备端录制安静场景“小云小云”数据进行微调，得到最终面向业务的模型。后续用户可在basetrain模型基础上，使用其他关键词数据进行微调，得到新的语音唤醒模型，但暂时未开放模型finetune功能。')

```
print(result[OutputKeys.TEXT])
```

基于模型的语义分块

Sequence Model with Self-Adaptive Sliding Window for Efficient Spoken Document Segmentation

总结

◆ 总结

挑战：企业数据格式多样和内容多样

原则：GIGO垃圾进垃圾出，数据质量的重要性

GIGO应对：严格筛选数据源，清理矛盾和不相关的数据

GIGO应对：多种文档解析工具，减少解析内容和样式出错（基于ocr和视觉的解决方案deepdoc+定制开发）

GIGO应对：文本分块，相同语义分块在一起，（基于规则的分块和基于语义的分块）

企业数据的智能处理